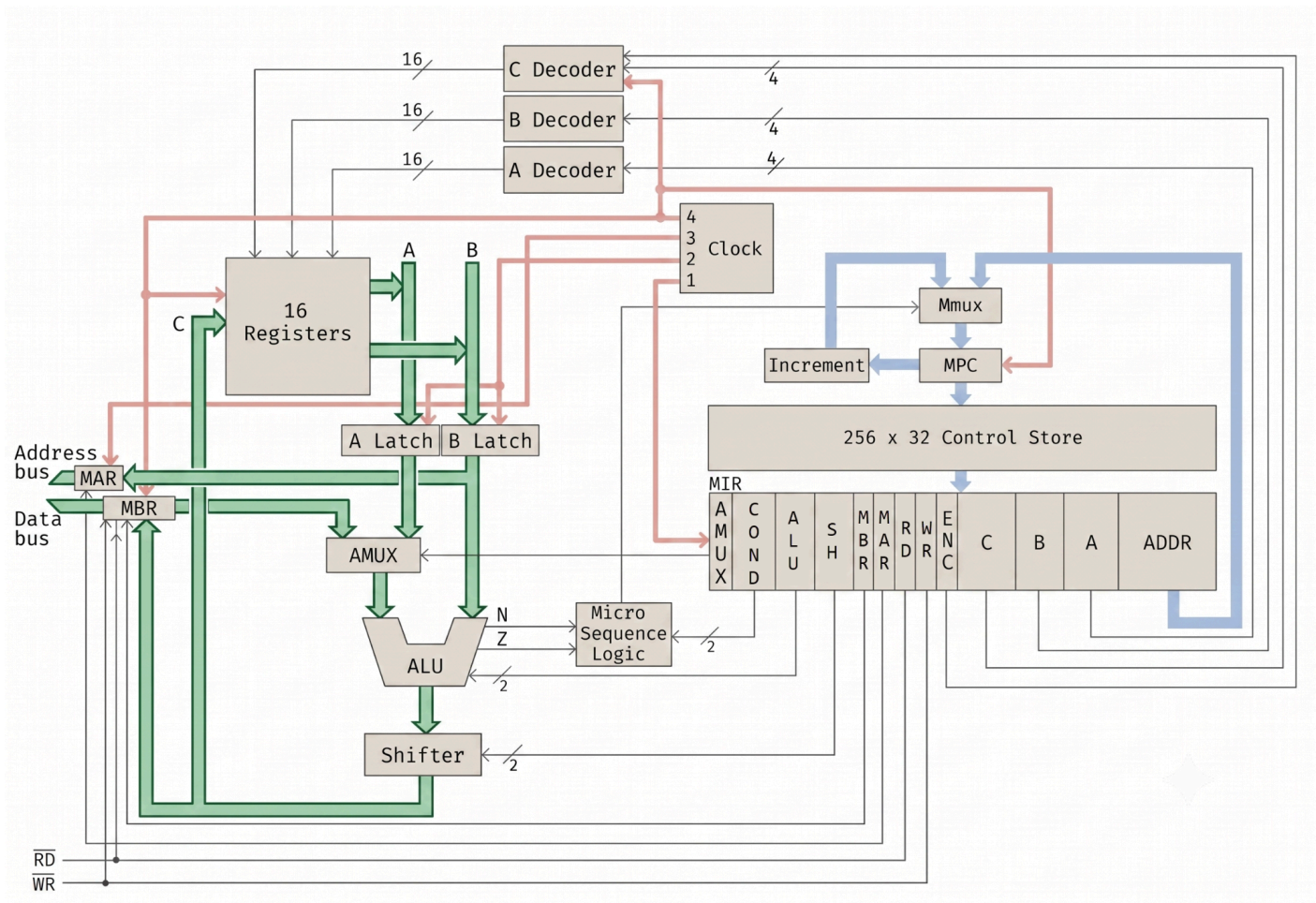


Fundamentos de Arquitectura de Computadoras

Arquitectura Horizontal



Con registros:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
PC	AC	SP	IR	TIR	0	+1	-1	AM ask	SM ask	Ax	Bx	Cx	Dx	Ex	Fx
					x00 00	x00 01	xFF FF	x0F FF	x00 FF						

Estructura del MIR

A M U X	C O N D	ALU	SH	M B R	M A R	R D	W R	E N C	C	B	A	ADDR

AMUX

0 = Buffer A
1 = MBR

COND

0 = No salta
1 = Si N==1
2 = Si Z==1
3 = Salta siempre

ALU

0 = A + B
1 = A & B
2 = A
3 = ~A

SH

0 = No desplaza
1 = alu >> 1
2 = alu << 1
3 = (no usado)

MBR, MAR, RD, WR, ENC

0 = No
1 = Si

Arquitectura Horizontal - Plantilla para microprogramación

[illegible]

Fundamentos de Arquitectura de Computadoras
Arquitectura Horizontal - Instrucciones Assembler

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxxxx	JPOS	Jump positive	if ac>=0 then pc:=x
0101xxxxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<=x<4095)
1000xxxxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

Fundamentos de Arquitectura de Computadoras

Arquitectura Horizontal - Microprograma

0: $mar := pc ; rd ;$	{ciclo principal}	40: $tir := lshift(tir) ;$ if n then goto 46 ;	{110x o 111x?}
1: $pc := pc + 1 ; rd ;$	{incrementa pc}	41: $alu := tir ;$ if n then goto 44 ;	{1100 o 1101?}
2: $ir := mbr ;$ if n then goto 28 ;	{salva y decodifica mbr}	42: $alu := ac ;$ if n then goto 22 ;	{1100 = JNEG}
3: $tir := lshift(ir + ir) ;$ if n then goto 19 ;		43: goto 0 ;	
4: $tir := lshift(tir) ;$ if n then goto 11 ;	{000x o 001x?}	44: $alu := ac ;$ if z then goto 0 ;	{1101 = JNZE}
5: $alu := tir ;$ if n then goto 9 ;	{0000 o 0001?}	45: $pc := band(ir , amask) ;$ goto 0 ;	
6: $mar := ir ; rd ;$	{0000 = LODD}	46: $tir := lshift(tir) ;$ if n then goto 50 ;	
7: $rd ;$		47: $sp := sp + (-1) ;$	{1110 = CALL}
8: $ac := mbr ;$ goto 0 ;		48: $mar := sp ; mbr := pc ; wr ;$	
9: $mar := ir ; mbr := ac ; wr ;$	{0001 = STOD}	49: $pc := band(ir , amask) ; wr ;$ goto 0 ;	
10: $wr ;$ goto 0 ;		50: $tir := lshift(tir) ;$ if n then goto 65 ;	{111, examina addr}
11: $alu := tir ;$ if n then goto 15 ;	{0010 o 0011?}	51: $tir := lshift(tir) ;$ if n then goto 59 ;	
12: $mar := ir ; rd ;$	{0010 = ADDD}	52: $alu := tir ;$ if n then goto 56 ;	
13: $rd ;$		53: $mar := ac ; rd ;$	{1111000 = PSHI}
14: $ac := mbr + ac ;$ goto 0 ;		54: $sp := sp + (-1) ; rd ;$	
15: $mar := ir ; rd ;$	{0011 = SUBD}	55: $mar := sp ; wr ;$ goto 10 ;	
16: $ac := ac + 1 ; rd ;$	{Nota: $x - y = x + 1 + no\ y$ }	56: $mar := sp ; sp := sp + 1 ; rd ;$	{1111001 = POPI}
17: $a := inv(mbr) ;$		57: $rd ;$	
18: $ac := ac + a ;$ goto 0 ;		58: $mar := ac ; wr ;$ goto 10 ;	
19: $tir := lshift(tir) ;$ if n then goto 25 ;	{010x o 011x?}	59: $alu := tir ;$ if n then goto 62 ;	
20: $alu := tir ;$ if n then goto 23 ;	{0100 o 0101?}	60: $sp := sp + (-1) ;$	{1111010 = PUSH}
21: $alu := ac ;$ if n then goto 0 ;	{0100 = JPOS}	61: $mar := sp ; mbr := ac ; wr ;$ goto 10 ;	
22: $pc := band(ir , amask) ;$ goto 0 ;	{realiza el salto}	62: $mar := sp ; sp := sp + 1 ; rd ;$	{1111011 = POP}
23: $alu := ac ;$ if z then goto 22 ;	{0101 = JZER}	63: $rd ;$	
24: goto 0 ;	{no se produce el salto}	64: $ac := mbr ;$ goto 0 ;	
25: $alu := tir ;$ if n then goto 27 ;	{0110 o 0111?}	65: $tir := lshift(tir) ;$ if n then goto 73 ;	
26: $pc := band(ir , amask) ;$ goto 0 ;	{0110 = JUMP}	66: $alu := tir ;$ if n then goto 70 ;	
27: $ac := band(ir , amask) ;$ goto 0 ;	{0111 = LOCO}	67: $mar := sp ; sp := sp + 1 ; rd ;$	{1111100 = RETN}
28: $tir := lshift(ir + ir) ;$ if n then goto 40 ;	{10xx o 11xx?}	68: $rd ;$	
29: $tir := lshift(tir) ;$ if n then goto 35 ;	{100x o 101x?}	69: $pc := mbr ;$ goto 0 ;	
30: $alu := tir ;$ if n then goto 33 ;	{1000 o 1001?}	70: $a := ac ;$	{1111101 = SWAP}
31: $a := ir + sp ;$	{1000 = LODL}	71: $ac := sp ;$	
32: $mar := a ; rd ;$ goto 7 ;		72: $sp := a ;$ goto 0 ;	
33: $a := ir + sp ;$	{1001 = STOL}	73: $alu := tir ;$ if n then goto 76 ;	
34: $mar := a ; mbr := ac ; wr ;$ goto 10 ;		74: $a := band(ir , smask) ;$	{1111110 = INSP}
35: $alu := tir ;$ if n then goto 38 ;	{1010 o 1011?}	75: $sp := sp + a ;$ goto 0 ;	
36: $a := ir + sp ;$	{1010 = ADDL}	76: $a := band(ir , smask) ;$	{1111111 = DESP}
37: $mar := a ; rd ;$ goto 13 ;		77: $a := inv(a) ;$	
38: $a := ir + sp ;$	{1011 = SUBL}	78: $a := a + 1 ;$ goto 75 ;	
39: $mar := a ; rd ;$ goto 16 ;			